Processes Board Notes (previous notes)

The process environment

The purpose of this section is to make sure that we are all working from a common background. This material will be review for many of you, but will be new for some of you. Details will be added in class.

Creating executable programs

Steps to create an executable program from C++ source files. Note: Most of these are hidden by the compiler/development environment. (I.e. the IDE) More detail in class.

1. Preprocessor - gets data from include files and organizes it. It also fills in all macros and defines in the source code.

2. Compile - translate source code into its machine language equivalent. (.o file for Linux, .obj for Windows)

   A .o file is not an executable program. Notes are recorded in the .o or .obj file to indicate externs (what it needs) and entry points. (what it has for other object files.)
3. Note: The first two steps occur to each source file in isolation. Namely, all of the .cpp files are translated independently. Important concept.

4. Link - the .o files, standard library files, user supplied library files are put together to create an executable.

   This is the point where you will see link errors indicating undefined externals and multiply defined entry points. Shared libraries are handled slightly differently. We will discuss in class. We will note the advantages of each type of library.

5. Run - the OS is directed to run the program. Mention command-line arguments.

Creating user defined libraries Start here 9/24/2025

- Libraries are both an organizational tool and a means of reusing software.
- They are files that contain a set of .o files and an index for accessing the functions within. Libraries are postfixed with a ".a".
- Note: you can also generate shared libraries which are postfixed with .so. These are equivalent to the Windows DLLs.

- If you are building using a make file or working at the command level, the "ar" (archive) command may be used to create libraries. Usage for the "ar" command is:

  ar -rv <Library name>.a <list of .o files>

  Note: ar - archive, r - means replace, v - means verbose.

- You can also create libraries from the Eclipse or other GUI. With these you don't have to think as much.
- We will talk about when shared libraries are better and when static libraries are better.

Example using a makefile

We will create a directory to hold the a function or two and create a library. We will then write code to exercise the functions. The following is the make file that we will use to create a library containing syserr.o (Note to me - this is in the library4 subdirectory.) Note: must have a tab in front of an action.

```
rcUnix.a : syserr.o
    ar -rv rcUnix.a syserr.o

syserr.o : syserr.cpp syserr.h
    g++ -g -c syserr.cpp
```

Below is a make file that uses the library we just built. The make file for the code will be:

```
ex3: ../library4/rcUnix.a ex3.o
    g++ ex3.o ../library4/rcUnix.a -o ex3

ex3.o: ex3.cpp
    g++ -c -g ex3.cpp
```

Note: that the library file must come after the .o files in the link line

We can also use Eclipse to generate libraries. We might later do this in class.

The main function

The main function is the starting point for the program. It has at least 3 prototypes in Linux/UNIX. These are:

```
int main();

int main( int argc, char *argv[] );

int main( int argc, char *argv[], char *envp[] );
```

The argc and argv arguments have been discussed in the past. envp allows you to access environmental variables. PATH, USER, and SHELL are examples of these. There is no argument that indicates how many environmental variables. How do you think we know this how many. Let's look at these. Note: the envp argument tends not to be used. (Dropped from Posix standard.) It still works in Linux. There are other ways to get at the environmental variables. For example, there is the global variable environ: "extern char **environ;" may be used to access this info. We will talk more in class. (The env command will display the set of environmental variables.)

If we return from the main, we terminate the program. Why an int value? If we fall off the end of the main function, we in effect return with a value of 0.

The exit calls

I will talk about 3 exit system calls.

void exit(int status);

> This function causes the program to exit (terminate) after cleaning up after itself. For example, it flushes and closes open files. The status is traditionally 0 to indicate that all is well and 1 or some other positive integer if there was a failure. Some people use a bunch of numbers to indicate different types of failure. Because of other system calls, it is good to keep that value of status between 0 and 255. There is access to this value as we will see later.

void _exit(int status);

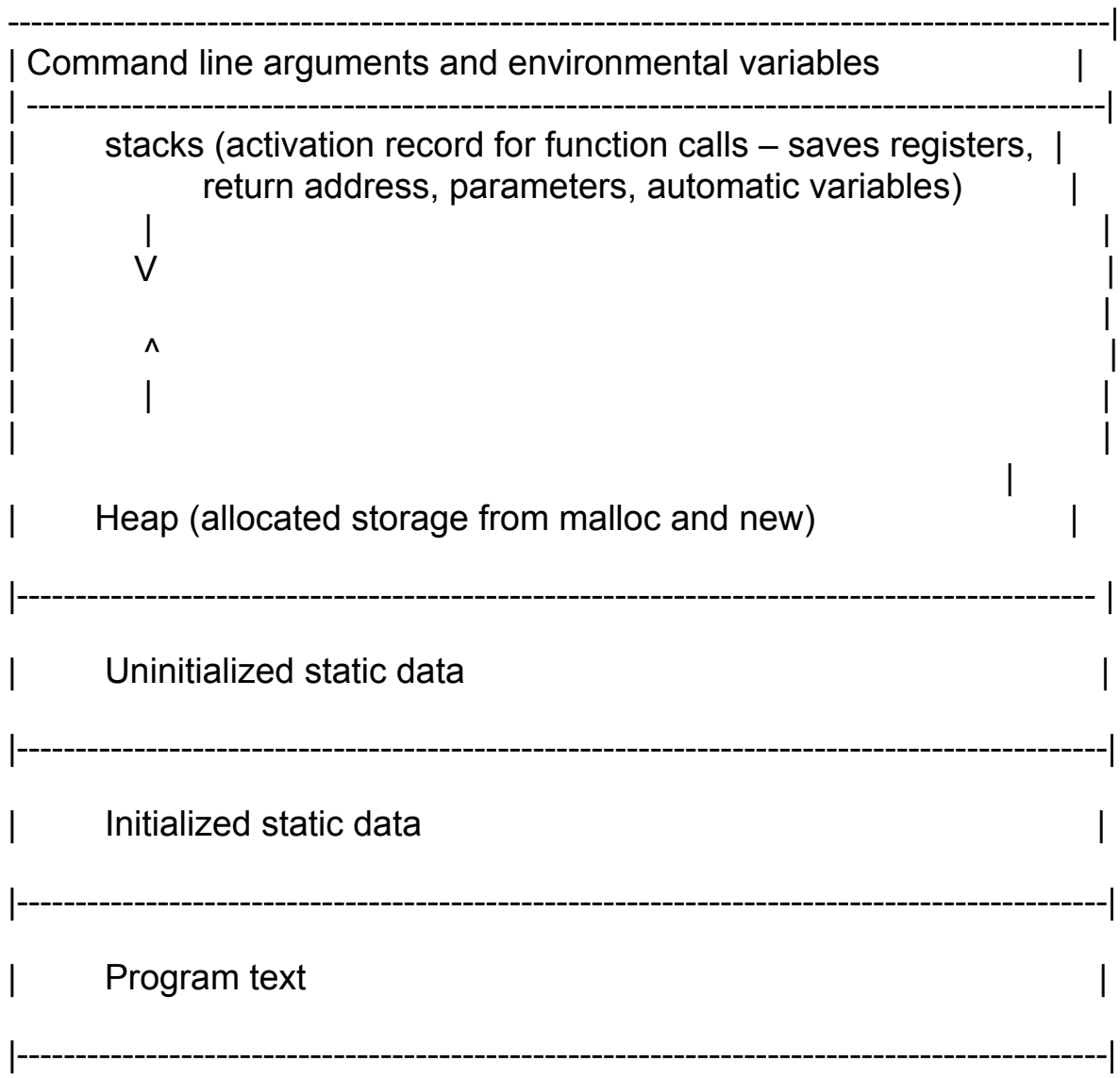
> This function exits with no cleanup. (E.g. buffers not flushed.) It is called when you want the program to terminate immediately. Status means the same as before. It also does not call the atexit functions. (See below) It is also useful with the fork call that we will talk about later.

void atexit(void (*func) (void));

This function is surprisingly in the **standard C library**. This function may be used to register up to 32 functions to be called when the program exits. The functions are called in the reverse order that they were registered. It is used to provide the ability to do any final cleanup before the program terminates. For example, a server may wish to send a goodbye message to its clients. **Let's try this in a program. Let's see what happens if the program aborts.**

Note: if your program aborts, the internal buffers will **not** be flushed. Why is this good to know?
Note: the command: env will display your environmental variables.

Layout of a running program in memory.



Since multithreading is built into Linux, there is a need for multiple stacks. Why? How is this supported? This has important implications with respect to storage for local variables.

Reminder to me: Mention shared libraries if I haven't done so earlier.

The system call

This call is very useful and simple. To call commands or execute programs from within a program you use the following:

```
int system( const char *string );
```

Returns -1 if could not schedule program, otherwise returns status information about the command (see waitpid later for how to interpret the status information).

This call is a cute way to execute any command from your program. For example, there is a mailx command that we discussed earlier. This call is one way to send email from your program. (Probably the easiest unless you are using someone's library.

You may not use the system call in labs or tests unless I give you permission. It makes things too easy and you won't get a true understanding of the systems calls.

Process information

Concepts:

- Process ID - a unique positive integer that identifies the process.
- The process that scheduled a given process is called its parent. The scheduled process is called its child.
- A program that is run from the shell has the shell as its parent.
- Why use a process ID rather than process name?
- Access to process IDs. Why do we need? For the sake of debugging and other calls to come.

```
#include <sys/types.h>
#include <unistd.h>
pid_t getpid( void );    // This gets the process's ID.
```

```
pid_t getppid(void);    // This gets the ID of the parent process.
```

Process state: The status of a process at a given time. Here are some of the process states.

New: process created.

Ready: process waiting for CPU time. I.e. to be scheduled.

Blocked: process waiting for an event. This can be as simple as waiting for user input.

Running: a processor is executing the process.

Done: process completes and resources are freed.

In your operating systems class, they probably drew state diagrams for processes in the system. We will not do this here.

ps Command

ps command: to display processes on system. Without any arguments, displays the processes for your session. If want to display everything on the system, use the following format:

ps -e (all processes) -f (generate a full listing)

Example: ps -ef | m

Note: if you use the -f option, the column headings mean:

- UID - the user ID of the process owner.
- PID - the process ID of the process listed.
- C - the percentage of CPU time recently used.
- TTY - "The terminal associated with the process." Not very useful
- TIME - the total CPU time used by the process.
- CMD - the command that launched the process.

Note: For most commands including this one, there are a lot of options. There is also a lot of variation in the options between different versions of UNIX. Of course,

there are not as many versions actively used as there used to be. Note on how to clean up eclipse if you terminate MobaXterm while running it.

The process environment.

- A bunch of system variables that describe the running environment for programs.
- printenv displays the environmental variables. So will the command env.
- Can be used for global information to support a programming environment.
- If we used a .bashrc file, the file may be used to set some of these when the shell starts up. Let's check out my shell.
- Let's take a look at some of the variables that bash uses.

  PATH - where the shell looks for programs when you type a name.

  HISTSIZE - size of history file in lines What do you think this is?
  PS1 The command prompt I have this now set up in my .bashrc file.

  SHELL The path name of the shell

You can view these by typing: echo $<name of environment variable>

You can create and set an environment variable on the command line by typing (this is for k-shell and bash shell. Other shells may have different formats):

DUCK = quack

export DUCK

or export DUCK=quack

Note: export means that it will stay around when you run other programs from the shell. How can we demonstrate this?

You can display the environmental variable DUCK from the command line by typing: echo $DUCK

You can display the entire environment by typing env. You can display the value of a single environmental variable by typing: echo $<env Var>

Take a look at the PATH variable and see why it can be a problem. Strange what was done!!! Any ideas why?

Note: different shells will have different rules for setting these variables. For example,

with C shell, we set DUCK by:

setenv DUCK quack

Programmatically can access environmental variables in a number of ways. We can do it through the parameters of the main function or though an external variable..

extern char **environ;

or

main((int argc, char *argv[]; char *env[])

Let's create an example for displaying the environment.

env command: used to display the environment. Let's try it.

getenv and putenv system calls. To get and set environmental variables. See the man pages for detail.

#include <stdlib.h>

char *getenv(const char *name) returns pointer to the value of variable

(NULL return if variable not found)

int putenv(char *string); sets an environmental variable. Form: <name>=<value>

Let's throw together an example of using genenv.

An interesting use of the putenv function is to change the time zone (TZ) that your system thinks it is using. That means that you can write an application where calls to get local time will be in a different time zone. Why would you want this?

Let's try **export** TZ='Asia/Kolkata' and see the effect on the date command. Careful with extra spaces. The command "date" will display the current date and time.

Note: putenv might be setenv on some other UNIX systems.

Lab3

Write a program to use putenv to change the TZ value to western time. (California time) Look up how to get the local time in your program and display it before and after the change. Why don't you see the time zone change after you exit the program?

Review Sending email. Don't use this for Fall 2025

The mailx command will allow you to send email with attachments. I am sending the mail to myself and the duckie. (The blank was put in may email address to foil spammers) You do it as follows:

Type the following to send tron1.cpp and tron2.cpp

mailx -s "test mailx" -A tron1.cpp -A tron2.cpp vmiller @ramapo.edu duckie@ramapo.edu

After you hit enter, you can type a message. Type as many lines as you want. When you are finished. Hit cntrl d.

The -A option means attachment. (Weird because -a means attachment when I used Google on xmail.) You list email destinations at the end. -s means subject line.

You can use this to send in your labs and tests.

Scheduling processes Start Here 10/1/2025

This is a very scary section for system administrators as you will see. In the past we have had a few student create "interesting" events which brought down the system. I will describe. Please be careful.

This section is devoted to process scheduling. This allows us to write interesting applications and understand code written by others.

The fork system call

- creates a clone of the calling process. I.e. It makes a copy of the parent including its data space and open file descriptors.
- Why would you want to create a clone? We will discuss in class.
  Note that most system level parameters are not changed for the new process.
- The data that is changed are things like the process ID, parent process ID, and

accounting data. Accounting data would be things like the total CPU time used.

- Important to realize that the child process has copy of parent's open file descriptors. You might need to clean up for both safety and lowering risk of using too many system resources. How can I do this safely?
- Below is the fork system call.

```
#include <sys/types.h>

#include <unistd.h>

pid_t fork(void); returns 0 if child, pid child if parent and -1 if error.
```

Examples:

- The following is a program that can generate a lot of children. (**Please don't do the test I will demonstrate**) (fork1.cpp) Note the order in which the child processes terminate. Here is where I discovered about setting the console to a fixed width: **<Window> - <Preference> - <Run/Debug> - <Console> Set Fixed width.** I set it to 80.
- Note the issues of running from eclipse. We don't see the final data.
- Also do fork2.cpp (this demonstrates that a child process has variables that are independent from the parent.) With the endl, we don't need to flush. (At least we don't if iostream is working correctly.)
- We might talk of kind of makefile that could be used to compile and link 3 fork projects. This is just to give you more of a feeling for make files. It assumes that syserr.o is available and does not have to be remade.

BE careful with the fork call. You can use up all the processes allowed to you and if no user limit is set, all the process IDs available to the system. This latter event force IT to do a power boot.

Lab4

Write a program which will generate a child. The child will have another child, and so on for 10 generations of children. The parent will display its PID and terminate. Each child will sleep for 1 second before it generates a child, display its PID, display its, parent's PID and terminate. Have the child announce its number that you will assign to represent its generation. **Please be very careful with this code. You can cause a real mess.**

The vfork system call

- vfork system call: just like fork except it does NOT make a copy of the parent's data area. Why was this important? This is now pretty much obsolete. **What do you think happened? (Now, with fork, a full copy of the parent's environment does not occur if nothing changes.) You can probably also guess why vfork is still around.** We will look at it, since it is still around and used. BTW, people do use it to make it very clear that they are only creating a clone to create a new process. Also, I have a question of whether the file descriptors are closed with fork.
- We will see that it can be used to support scheduling of a new program. The fork call is now good enough to support this.
- On most UNIX systems you must be very careful not to do anything in the child from vfork that might mess up the parent process. It seems that Linux requires this level of care.
- If vfork child process exits should use _exit function to terminate. But, if you use exit, it will work in Linux. So will return from the main.

  prototype:

  int vfork();

See fork3.cpp Why do I have flush calls? Check what happens if I don't do an _exit or exit but a return instead. It is not an issue any more, but good to check.

Note: vfork used to be very important when the parent is a large process. **Using fork to make a copy of the parent used to be incredibly expensive if the only purpose of doing this is to launch another program and the parent is large.** We will talk more about this in class. The issue is the cost of making a clone is no longer an issue for Linux. **Recall vfork is not that important any more. If you use a UNIX system other than Linux, it would be smart to check on this.**

I have an example from industry where fork caused big problems when it was used. This was before the change made to fork.

The exec system calls

- The exec systems calls. Used to replace one process by another.
- There are at least 6 forms of this call depending on what option you choose. The following are the various prototypes for the calls.

  int execl(const char *name, const char *arg0, const char *arg1, ..., const char *argn, (char *)0)

int execle(const char *name, const char *arg0, const char *arg1, ..., const char *argn, (char *)0, const char *envp[])

int execlp(const char *file, const char *arg0, const char *arg1, ..., const char *argn, (char *)0)

int execv(const char *name, const char *argv[];)

int execvp(const char *file, const char *argv[];)

int execve(const char *name, const char *argv[], const char *envp[])

- All functions should only return if error in scheduling the program. So, it is irrelevant to talk about the return value. Why?
- The letters in the name of the function tells us what it does.
    - l - means argument list
    - e - means environment. This means that the environmental variables will be passed to the new process by the calling program.
    - p - means uses the PATH environmental variable to locate the first instance of the executable file name.
    - v - stands for vector of pointers to the arguments. It is a vector in the older sense. Namely, it is an array. How do they get the size?

Note: the process launched by one of these functions inherits open file descriptors and has the same PID as its parent.

The wait, waitpid system calls.

Note that there are other waits, but the ones described here are more universal.

There is a BIG implication of having a wait function in our system. It implies that information about a terminated child process must be kept after the child has terminated. For such cases the child process is referred to as a zombie. This can have an impact on the number of available process IDs. I will discuss how this had an impact on code I was developing a number of years ago. I will also discuss how to deal with

this issue.

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait( int *stat_loc );

pid_t waitpid( pid_t pid, int *stat_loc, int options );
```

- Both functions return the process ID of the terminated child.
- They return -1 with errno = ECHILD if there are no remaining child processes running.

- stat_loc returns information on why the process terminated. Various parts of the value given to stat_loc provides information on the reason that the program terminated. We could go through the bits of the value of stat_loc, but there are macros to analyze its values. If there are macros, why is it always better to use these? (Recall that the system call returns this.) The macros are:

  WIFEXITED(status) - true if normal termination

  WEXITSTATUS(status) 8 bit argument of exit function. Note the implication on the return from main and the exit call.

  WIFSIGNALED(status) - true if child terminated due to signal.

  WTERMSIG(status) - the signal number. We will talk of signals later.

  WIFSTOPPED(status) - true if child stopped. Again we will talk more about this later.

  WIFCONTINUED(status) - returns true if the child process was restarted. (Need SIGCONT for this)

- pid In waitpid indicates the manner in which this function will be handled. So, it gives us more options than the wait call.
  - If pid is equal to -1, status is requested for any child process. In this respect, waitpid is then equivalent to wait.
  - If pid is greater than zero, it specifies the process ID of a single child process for which status is requested.
  - If pid is equal to zero, status is requested for any child process whose process group ID is equal to that of the calling process.

- If pid is less than -1, status is requested for any child process whose process group ID is equal to the absolute value of pid.

- The options argument is constructed from the bit wise **inclusive or** of zero or more of the following flags, defined in the header <sys/wait.h>:
  - WNOHANG - The waitpid function does not suspend execution of the calling process if status is not immediately available for one of the child processes specified by pid. The return value is 0 if there were no zombies.
  - WNOWAIT - Keep the process whose status is returned in stat_loc in a waitable state. The process may be waited for again with identical results.
  - WUNTRACED - (Don't worry about this one.) If the implementation supports job control, the status of any child processes specified by pid that are stopped, and whose status has not yet been reported since they stopped, are also reported to the requesting process. That is the process calling waitpid:

  See examples execl, child, vforkexec (no need for vfork under most circumstances. We will recode example to demo it working.) Note: sometimes that final output will be in the wrong order. Why??? That is the error from wait will be reported before the termination message from cout.
  See waitpid example.

Lab 5

Write a program that will accept the names of 3 processes as command-line arguments. (This can be a single program that you list 3 times in the command line.) Each of these processes will run for as many seconds as: (PID%10)*2+3 and terminate. The parent process will reschedule each child 4 times before giving up. When all children have been rescheduled 4 times, the parent will terminate.